Milestone 2 — Plano de Métricas de Produto (ISO/IEC 25010) e Relatório de Análise Estática

Campo	Valor
Disciplina	Qualidade de Software — TADS, IFRS Campus Rio Grande
Professor	Me. Juan Carlos Quevedo Weimar — 2026.2
Aluno	Roberto Fabiano Cunha da Silva
Entrega	12/11/2026 (Milestone 2 do P2)
Objeto de estudo	projeto de código aberto **htop** — `github.com/htop-dev/htop`
Commit analisado	`8ff043d2429b0bb5a8c093d1442c3a1534603381` (20/09/2026) — o mesmo commit do Milestone 1, para que os números sejam comparáveis
Ferramenta	SonarQube Cloud (SonarCloud), projeto `robertoSilva-ux_htop` — https://sonarcloud.io/dashboard?id=robertoSilva-ux_htop
Fonte auditada (fork)	https://github.com/robertoSilva-ux/htop — fork do repositório oficial, fixado no commit `8ff043d`

---

1. Escopo e método

O Milestone 1 concluiu que o htop tem **maturidade de produto alta e maturidade de processo baixa**, e isolou por Pareto um recorte de 11 arquivos que concentram 100% das CVEs do projeto. Este milestone instrumenta exatamente esse recorte: sai do diagnóstico de processo (ISO/IEC 12207 + MR-MPS-SW) e passa à avaliação empírica do artefato (ISO/IEC 25010), na forma exigida pelo enunciado — **três características críticas, cada uma com métrica quantitativa e meta**, medidas por análise estática de código.

Face de qualidade medida: interna (visão do desenvolvedor, medida estática no código-fonte). Qualidade externa (execução) e em uso (contexto real de trabalho) não são medidas neste milestone; as métricas correspondentes ficam definidas e reservadas ao relatório consolidado.

1.1 Como a análise foi produzida

Etapa	Comando / artefato
Obtenção do	`git clone https://github.com/robertoSilva-ux/htop.git && git checkout 8ff043d` (fork

Etapa	Comando / artefato
código	do repositório oficial; o commit é idêntico ao do upstream)
Compilação	`./autogen.sh && ./configure && make` — gcc 16.2.1, ncursesw 6.6, container Fedora via podman rootless
Base de compilação	`compile_commands.json` → 86 unidades `.c` válidas (após descarte de `conftest.c` e sondagens do `configure`)
Análise	`sonar-scanner 7.1.0` com `sonar.cfamily.compile-commands` (CFamily + taint analysis)
Coleta de resultados	API pública do SonarCloud (`/api/measures/component`, `/api/issues/search`, `/api/qualitygates/project_status`)

O banco de compilação foi usado porque o `build-wrapper` 6.86 falhou em registrar o campo `files`; para C/C++ o `compile_commands.json` é o mecanismo suportado pelo SonarCloud e produz o mesmo resultado, com a vantagem de ser reexecutável fora do wrapper.

1.2 Universo medido e limites

Dimensão	Valor
Arquivos `.c`/`.h` no repositório	284 (132 `.c`)
Linhas físicas em `.c`/`.h`	49.241 (40.752 não brancas)
Arquivos `.c` efetivamente analisados	**86 de 132 (65%)**
Linhas de código (ncloc) medidas pelo Sonar	**37.458** em 293 arquivos monitorados
Funções com métricas	894

Por que 86 e não 132: os 46 arquivos restantes pertencem às camadas de plataforma (`darwin/`, `freebsd/`, `netbsd/`, `openbsd/`, `dragonflybsd/`, `solaris/`, `pcp/`, `unsupported/`) e não são compilados em Linux. O Sonar os contabiliza em linhas, mas sem complexidade nem funções — logo, **toda afirmação deste relatório sobre complexidade, dívida e defeitos vale para o código Linux**, explicitamente o que roda na máquina do usuário que este relatório representa. É uma limitação declarada, não um erro de medição.

---

2. Matriz de requisitos não funcionais → ISO/IEC 25010

ID	Requisito não funcional	Característica (ISO 25010)	Subcaracterística
RNF-01	Ler `/proc/<pid>/{stat,status,cgroup,environ}` de processos de outros usuários sem corromper estado	Segurança	Integridade / Confidencialidade
RNF-02	Um `htoprc` hostil não pode provocar leitura fora de limites nem execução de comando	Segurança	Integridade
RNF-03	Consumo de memória baixo e estável durante execução longa	Eficiência de desempenho	Utilização de recursos
RNF-04	Atualização da lista de processos sem travar a interface	Eficiência de desempenho	Comportamento temporal
RNF-05	Continuar operando diante de `/proc` parcialmente ilegível	Confiabilidade	Tolerância a falhas
RNF-06	Restabelecer estado íntegro após crash (crash handler grava relatório)	Confiabilidade	Recuperabilidade
RNF-07	Alterar um medidor/coluna sem quebrar os demais	Manutenibilidade	Modificabilidade
RNF-08	Diagnosticar causa-raiz de defeito a partir do código e do crash report	Manutenibilidade	Analisabilidade
RNF-09	Executar suíte automatizada de regressão sobre o comportamento	Manutenibilidade	Testabilidade
RNF-10	Compilar e rodar em Linux, macOS e BSDs	Portabilidade	Adaptabilidade
RNF-11	Instalar por pacote da distribuição, sem dependências exóticas	Portabilidade	Facilidade de instalação
RNF-12	Interface legível e operável por teclado, com proteção contra erro do usuário	Usabilidade	Operabilidade / Proteção contra erro

---

3. As três características críticas (justificadas por risco)

A escolha não é por simpatia nem por facilidade de medição: cada uma responde a um risco já materializado ou estrutural do produto.

3.1 Segurança (RNF-01, RNF-02) — prioridade 1

É a única característica com **evidência de dano real**: duas CVEs no histórico (`CVE-2008-5076`, renderização de nomes de processo sem sanitização; `CVE-2024-37676`, acesso fora dos limites em `Header_populateFromSettings` alcançável por `htoprc` adulterado). O produto processa entrada não confiável por definição — lê dados de processos alheios e um arquivo de configuração que o usuário pode editar (ou que pode chegar adulterado). Metade dos monitores de sistema é executada com `sudo` para ver todos os processos, o que eleva a consequência de qualquer falha de memória.

Medição de hoje: 12 vulnerabilidades BLOCKER de injeção (taint analysis) concentradas nos mesmos arquivos do recorte do M1.

3.2 Manutenibilidade (RNF-07 a RNF-09) — prioridade 2

C de 40 mil linhas não brancas, com 894 funções, **cobertura publicada de 0,0%** e dívida técnica de 645 minutos. O projeto tem um único arquivo de especificação de teste (`test_spec.lua`, 687 linhas) para 49 mil linhas de código. A manutenibilidade é o que explica por que as CVEs se concentram onde se concentram: sem teste dedicado nos parsers, a regressão só é descoberta em produção.

3.3 Eficiência de desempenho (RNF-03, RNF-04) — prioridade 3

É a **missão declarada do produto** — o `CONTRIBUTING.md` exige que o htop "keep a small footprint". Um monitor de processos que consome recursos demais é inútil por definição, e essa é a razão pela qual o htop existe em relação ao `top` tradicional. É também a característica em que o produto **já é forte** — o que a torna útil como contraponto: nem toda característica precisa de plano de melhoria.

Descartadas com justificativa: Adequação funcional (a funcionalidade é madura e estável há duas décadas; o risco está na entrada, não na função); Compatibilidade (interoperabilidade não é eixo do produto); Portabilidade (já coberta por 20 jobs de CI em 7 plataformas — medir de novo não acrescenta informação).

---

4. Plano de métricas de produto

ID	Métrica	Fórmula / definição	Ferramenta	Meta	Gate	Frequência
M-01	Vulnerabilidades de injeção em parsers	nº de achados taint BLOCKER/CRITICAL em `Settings.c`, `LinuxProcessTable.c`, `Platform.c`	SonarCloud (CFamily)	**0**	Bloqueia merge em PR que toque os parsers	por PR
M-02	Hotspots de segurança revisados	`security_hotspots_reviewed`	SonarCloud	**100%**	Sim	por release
M-03	Cobertura de ramos nos	ramos exercitados / ramos	gcov/lcov + Sonar	≥ **80%** no recorte	Sim (ratchet:	por PR

ID	Métrica	Fórmula / definição	Ferramenta	Meta	Gate	Frequência
	módulos críticos	totais		M1	nunca cair)	
M-04	Complexidade ciclomática por função	V(G) de McCabe por função	Sonar (`complexity`)	média **≤ 10**, nenhuma função **> 30**	Aviso > 20, bloqueio > 30	por release
M-05	Complexidade cognitiva por função	soma de aninhamentos e quebras de fluxo	Sonar (`cognitive_complexity`)	**≤ 15**	Aviso	por release
M-06	Densidade de duplicação	linhas duplicadas / linhas totais	Sonar	**≤ 3%**	Sim	por release
M-07	Índice de dívida técnica	minutos de remediação estimados	Sonar (`sqale_index`)	**≤ 5%** do esforço total	Aviso	por release
M-08	Densidade de defeitos	defeitos abertos / 1.000 ncloc	Sonar + issues	**≤ 1,0**	Aviso	mensal
M-09	Memória residente	RSS com 500 processos	`ps`//`proc` em bancada	**≤ 20 MB**	Não (informativo)	por release

Regra de leitura: M-01 a M-08 são métricas de **qualidade interna** — medida estática, antecipam o defeito; M-09 é de **qualidade externa** — medida com o programa em execução, confirma o comportamento. As demais métricas de qualidade externa e em uso (tempo de inicialização ≤ 300 ms por `hyperfine`, taxa de falha ≤ 0,1 por 1.000 execuções, cobertura de plataformas no CI) estão definidas e são acompanhadas no relatório consolidado, onde a avaliação dinâmica é executada.

---

5. Resultados da análise estática (qualidade interna)

5.1 Panorama

Métrica	Valor medido	Leitura
ncloc	37.458	base analisada

Métrica	Valor medido	Leitura
Funções	894	—
Complexidade ciclomática total	4.693	média **5,2 por função**
Complexidade cognitiva total	5.363	média 6,0 por função
Duplicação	**6,8%**	acima da meta M-06 (≤ 3%)
Dívida técnica	**645 min (≈ 10 h 45)**	remediação estimada pelo Sonar
Densidade de comentários	7,9%	baixa, típica de C de sistema
Cobertura de testes	**37,5% linhas / 23,5% ramos**	medida nesta auditoria; a CI publica 0% (NC-05)
Rating de confiabilidade	**E (5)**	pior faixa
Rating de segurança	**E (5)**	pior faixa
Rating de manutenibilidade	**A (1)**	melhor faixa

A contradição aparente — manutenibilidade A com confiabilidade e segurança E — é o resultado mais informativo do relatório: o código é **bem organizado** (funções curtas em média, nomenclatura consistente, estilo guiado por lint) mas **frágil onde importa** (parsers de entrada não confiável). É a assinatura de um projeto que investiu em convenção e automação de estilo, e não em teste de comportamento.

Nota sobre a cobertura: o dashboard publica `coverage = 0.0`, mas o projeto **tem** instrumentação — o `Makefile.am` define o target `coverage:` que recompila com `-fprofile-arcs -ftest-coverage`, e nenhum workflow de CI o utiliza. Executando o binário instrumentado, nenhum `.gcda` é gravado: o htop encerra por `_exit(0)` (`CRT.c:972,983`) e `_exit` não executa handlers de `atexit`, que é onde o gcov grava. Interceptando `_exit` por `LD_PRELOAD` e navegando a interface dirigidamente em pty, a cobertura medida é **37,5% de linhas e 23,5% de ramos** — um **piso**, não a suíte oficial (`test_spec.lua` não roda: `rote` não tem *rock* para Lua 5.5). O número prova que a cobertura do htop não é 0%: é **não medida pela CI**. Detalhamento da instrumentação e do contorno do `_exit` no relatório consolidado.

5.2 Distribuição das não-conformidades

73 issues abertas, todas em código existente:

Severidade	Bugs	Code Smells	Vulnerabilidades	Total
BLOCKER	4	26	**12**	**42**
MAJOR	3	5	13	21
MINOR	0	0	10	10
Total	**7**	**31**	**35**	**73**

Triagem obrigatória dos 42 BLOCKER: 26 são code smells de estilo (`c:S128` fall-through não anotado em `switch`, 9 ocorrências; `c:S1219` rótulo enganoso, 5; `c:S1909`/`c:S999` `goto`, 6; `c:S1912` função não reentrante, 3; `c:S912` efeito colateral em operando lógico, 3). São dívida de legibilidade, não risco de falha. Os 16 restantes é que sustentam o veredito: **12 vulnerabilidades + 4 bugs de acesso fora de limites**.

Esta é uma não-conformidade *do próprio instrumento* que precisa constar: tratar "BLOCKER" como sinônimo de "crítico para o negócio" produz um número inflado por um fator de 2,6× (42 → 16).

5.3 Vulnerabilidades BLOCKER — onde estão

Arquivo	Linha	Achado
`Settings.c`	359, 385	path injection via `fileName` em `Settings_read()` — `S2083`
`Settings.c`	908	path injection via `this` em `Settings_write()` — `S2083`
`Settings.c`	1106	path injection via `configDir` em `Settings_new()` — `S2083`
`linux/Compat.h`	38	path injection via `pathname` em `Compat_openat()` — `S2083`
`linux/LinuxProcessTable.c`	566, 570, 572, 574	array index injection via `ptr` em `LinuxProcessTable_readStatusFile()` — `S9340`
`linux/LinuxProcessTable.c`	954	array index injection via `buffer` em `LinuxProcessTable_readCGroupFile()` — `S9340`
`linux/Platform.c`	552	array index injection via `size` em `Platform_getProcessEnv()` — `S9340`
`linux/LinuxProcess.c`	157	command injection via `this` em `LinuxProcess_updateIOPriority()`

Arquivo	Linha	Achado
		— `S2076`

Cinco arquivos. Nenhum fora do recorte do Milestone 1. O Pareto de Juran que o M1 estabeleceu por análise de CVEs se confirma por medição independente: 100% das vulnerabilidades BLOCKER de injeção estão nos mesmos arquivos que o M1 apontou — `Settings.c`, `LinuxProcessTable.c`, `Platform.c` e `LinuxProcess.c` (os quatro `.c` entre os 86 analisados) mais o header `Compat.h`.

5.4 Complexidade no recorte do M1

Arquivo	ncloc	Funções	V(G) total	V(G)/função	Cognitiva	Code smells
`linux/LinuxProcessTable.c`	1.326	35	405	**11,6**	**617**	0
`Settings.c`	1.018	34	325	**9,6**	450	1
`linux/Platform.c`	948	30	163	5,4	234	2
`CRT.c`	1.397	17	69	4,1	66	0
`Action.c`	848	54	163	3,0	152	1
`CommandLine.c`	464	6	112	**18,7**	165	1
`Row.c`	462	26	121	4,7	116	0
`OpenFilesScreen.c`	272	8	60	7,5	46	1
`RichString.c`	217	18	46	2,6	43	0
`TraceScreen.c`	169	6	35	5,8	42	0
`generic/UnwindPtrace.c`	113	—	—	—	—	não compilado em Linux
Soma (10 compilados)	**7.121**	234	1.499	6,4	1.931	6

(`generic/UnwindPtrace.c` entra no recorte do M1 mas não é compilado em Linux: aparece com 113 ncloc e sem métricas.)

Duas funções-âncora violam a meta M-04 (V(G)/função ≤ 10): `CommandLine.c` (18,7) e `linux/LinuxProcessTable.c` (11,6). Não por acaso, são os dois arquivos do recorte com achados de segurança — complexidade alta é onde o defeito se esconde.

5.5 Quality Gate

O projeto está associado ao gate **Sonar way**. Resultado: **ERROR** — duas das quatro condições falham:

Condição	Limiar	Valor medido	Status
`new_reliability_rating`	≤ 1 (A)	**5 (E)**	**ERROR**
`new_security_rating`	≤ 1 (A)	**5 (E)**	**ERROR**
`new_maintainability_rating`	≤ 1 (A)	1 (A)	OK
`new_security_hotspots_reviewed`	≥ 100%	100%	OK

Por que reprova: o repositório foi analisado com o SCM habilitado (revisão `8ff043d` registrada no SonarCloud). Sem uma análise anterior, todo o código entra como "código novo", então as condições `new_*` recaem sobre o legado inteiro e expõem exatamente o que a seção 5.2 mediu: confiabilidade e segurança em E.

A ressalva conceitual permanece: o gate padrão mede o **delta**, não o produto. A partir da segunda análise, "código novo" volta a ser apenas o que mudou desde a última — e um PR que não toque nos parsers passará com gate verde, independentemente da dívida de 42 BLOCKER continuar lá. Gate verde, nesse formato, significa "o que você acabou de escrever não piorou a média", não "o produto está conforme". Um gate que reflita o produto exige condição sobre o código **total** (ex.: `security_rating` geral ≤ 2), o que o plano de métricas propõe em M-01 e M-03.

Nota metodológica: a análise sem base de compilação (*Automatic Analysis*) encontra 17 das 35 vulnerabilidades — os achados de taint nos parsers de `/proc` e `htoprc` desaparecem. Para auditoria de segurança em C, a base de compilação não é detalhe de infraestrutura: é o que determina se o defeito é encontrado. Comparativo completo no relatório consolidado.

---

6. Casos de teste caixa-preta (EP/BVA, IEEE 829)

Técnicas das Aulas 07/08 aplicadas às entradas reais do htop: partições de equivalência (EP) e análise de valor limite (BVA) sobre as três superfícies de entrada — arquivo de configuração, linha de comando e `/proc`. Seis casos, selecionados por risco, no formato IEEE 829.

ID	Cenário	Pré-condição	Passos	Resultado esperado	Técnica
CT-01	`htoprc` válido	`~/.config/htop/htoprc` com `fields=0 48 49 46 47 2 1 1 1 1 1`	executar `htop`	inicia sem aviso, colunas conforme config	EP (partição válida)
CT-02	`htoprc` com campo fora da faixa	mesma config, `fields=0 999`	executar `htop`	ignora o valor inválido e usa o padrão; sem crash	BVA (limite superior + 1)
CT-03	`htoprc` com linha de	arquivo com `fields=`	executar `htop`	linha truncada, sem	BVA (limite

ID	Cenário	Pré-condição	Passos	Resultado esperado	Técnica
	8.192 bytes	seguido de 8.192 caracteres		overflow; processo não aborta	do buffer)
CT-04	`-p` com PID inexistente	PID 4.194.305 (acima de `PID_MAX`)	`htop -p 4194305`	mensagem de erro, saída limpa, sem crash	BVA (limite do domínio)
CT-05	`/proc/<pid>/status` truncado	arquivo lido pela metade (simulado com `LD_PRELOAD`)	abrir a tela de detalhes	campos ausentes exibidos como vazio, sem leitura fora de limites	EP (entrada parcial)
CT-06	`/proc/<pid>/cgroup` com linha > 4.096 bytes	cgroup name muito longo	abrir a coluna CGROUP	valor truncado, sem overflow	BVA (limite do buffer)

Prioridade de execução: CT-02 a CT-06 exercitam exatamente os pontos onde o Sonar apontou array index injection e path injection — são os casos que devem entrar primeiro na suíte, e são os que hoje não existem. CT-01 é o caso de controle: garante que a suíte distingue comportamento correto de falha.

---

7. Registro de não-conformidades (severidade × prioridade)

Severidade = impacto técnico; prioridade = urgência de correção. As duas dimensões são independentes (Aula 03).

ID	Não-conformidade	Evidência	Severidade	Prioridade
NC-01	Path injection em `Settings_read`/`Settings_write`/`Settings_new` — 4 achados BLOCKER	`Settings.c:359,385,908,1106`	**Crítica**	**Alta**
NC-02	Array index injection nos parsers de `/proc` — 6 achados BLOCKER	`LinuxProcessTable.c:566-574,954`; `Platform.c:552`	**Crítica**	**Alta**
NC-03	Command injection em `LinuxProcess_updateIOPriority`	`LinuxProcess.c:157`	**Crítica**	**Alta**
NC-04	Path injection em `Compat_openat`	`Compat.h:38`	Alta	Média
NC-05	Cobertura não medida pela CI: publica 0% contra **37,5% medido** nesta auditoria; `make coverage` existe mas quebra no `_exit(0)`	`coverage = 0.0`; seção 5.1	**Crítica**	**Alta**
NC-06	Duplicação 6,8% — 2,3× acima da meta M-06	`duplicated_lines_density`	Média	Média

ID	Não-conformidade	Evidência	Severidade	Prioridade
NC-07	2 arquivos do recorte acima da meta de V(G) (18,7 e 11,6)	`CommandLine.c`, `LinuxProcessTable.c`	Média	Média
NC-08	Quality Gate mede apenas o delta (`new_*`): reprova a primeira análise e pode ficar verde depois sem que a dívida seja paga	`project_status`: 2 de 4 condições em ERROR	Alta (processo)	Média

As não-conformidades de estilo (26 code smells BLOCKER), de supply chain (Actions não fixadas por hash) e de cobertura de auditoria (46 arquivos de plataforma fora da análise) estão levantadas e entram no relatório consolidado, onde a severidade é confrontada com o custo da correção.

---

8. Conclusão e ciclo PDCA

Plan — as três características críticas estão definidas com métrica e meta numérica (seção 4), ancoradas no risco demonstrado no M1.

Do — a base existe: CI obrigatório em 20 jobs, CodeQL, Coverity, sanitizers, style guide aplicado por lint. O htop não é um projeto descuidado.

Check — a medição mostra onde o cuidado não chegou: **12 vulnerabilidades BLOCKER de injeção, todas em 5 arquivos, todos dentro do recorte do M1**; cobertura publicada 0% contra **37,5% medidos**; duplicação 2,3× acima da meta; confiabilidade e segurança em **E**; Quality Gate em **ERROR** nas duas condições de rating.

Act — ações verificáveis, em ordem de retorno:

1. **Publicar cobertura no CI** — a instrumentação já existe no `Makefile.am`; falta corrigir a saída por `_exit(0)` (chamar `__gcov_dump()` no handler) e rodar `gcovr` no workflow, enviando o XML ao Sonar. Fecha NC-05 e habilita M-03.
2. **Escrever os 5 casos de teste de maior risco** (CT-02 a CT-06) antes de qualquer correção — teste que falha primeiro, correção depois. Fecha parcialmente NC-02.
3. **Corrigir as 12 injeções** com validação de índice e de caminho na fronteira de entrada, não com patch pontual (o precedente do CVE-2024-37676 mostra que o endurecimento estrutural funcionou, o patch não).
4. **Adicionar condição de código total ao gate** (`security_rating ≤ 2` sobre o repositório, não só sobre o delta), para que a dívida acumulada não saia do radar assim que a primeira análise estabelecer a baseline. Fecha NC-08.

O que este relatório não pode afirmar: que o htop seja inseguro em uso. As 12 vulnerabilidades são achados de análise estática de fluxo de dados (taint), que exigem confirmação por execução — é exatamente o que a seção 6 propõe e o que ainda não foi feito. Análise estática aponta o caminho; só o teste dinâmico fecha o caso. Todos os números deste relatório foram obtidos por execução das ferramentas citadas sobre o commit indicado; nenhum valor foi estimado.

---

9. Fontes e evidências

- **Repositório:** fork auditado `github.com/robertoSilva-ux/htop`, commit `8ff043d` (20/09/2026) — código, `CONTRIBUTING.md`, `TESTPLAN`, `test_spec.lua`, `.github/workflows/`. Origem: `github.com/htop-dev/htop`.
- **SonarCloud:** projeto `robertoSilva-ux_htop` (vinculado ao fork) — https://sonarcloud.io/dashboard?id=robertoSilva-ux_htop — revisão `8ff043d`, medidas, issues e quality gate consultados em 09/11/2026.
- **Artefatos locais da análise:** `compile_commands.json` (86 unidades), `sonar_metrics.json`, `per_file.json`, `issues_all.json`, `coverage.xml`.
- **Normas:** ISO/IEC 25010 (modelo de qualidade de produto), ISO/IEC 25000 (SQuaRE), ISO/IEC 12207 (processos de ciclo de vida), MR-MPS-SW (SOFTEX).
- **Material da disciplina:** Aulas 03 (métricas e não-conformidades), 05 (MPS.BR/CMMI), 06 (ISO 25010), 07 (V&V, IEEE 829), 08 (caixa-preta/caixa-branca, McCabe).
- **Milestone 1:** `M1-htop-mapeamento-processos.pdf` — mapeamento ISO 12207, diagnóstico MR-MPS-SW e recorte de Pareto.